

README SOKOBAN

TEMA 1 IA

Voi începe prin a descrie mai întâi implementarea algoritmului Learning Time A*(LRTA*). Am început prin a studia pseudocodul din cadrul cursului după care am început prin a modifica codul din laboratorul 1. De asemenea, am folosit aceste referințe pentru LRTA* [1] și pentru Beam Search [2].

LRTA*

La partea de LRTA* am modificat și din punct de vedere conceptual întrucât nu mai exista o frontieră și am un singur nod curent și de asemenea iau decizii pas cu pas pe măsura ce înaintez. La partea de euristica, o combin pe cea inițială cu o memorie adaptivă pe care o actualizez la fiecare pas fiind o diferență față de algoritmul clasic de A* unde nu modificăm deloc valoarea euristicii.

La fiecare pas din starea curentă generez toți succesorii iar pentru fiecare succesor obțin valoarea asociată, dacă nu există o inițializez. După care din fiecare mutare aleg cel mai bun succesor și actualizez valoarea stării curente. Algoritmul se oprește atunci când am depășit numărul de pași acceptat sau am ajuns într-o stare considerată finală. Costul este stocat într-un dicționar H unde mapez fiecare hărți un cost practic estimarea curentă învățată.

Practic, am adăugat algoritmul de A* ideea de generare a succesorilor, evaluează și actualizarea funcției de cost a euristicii și memorarea drumului.

Beam Search

La implementarea algoritmului Beam Search am pornit de la ideile din curs prezentate în cadrul algoritmului Local Beam Search. Am efectuat câteva modificări.

Am pornit dintr-o singură stare și nu din k generate aleator ca în documentul prezentat la curs. De asemenea, am evitat re-expandarea stărilor prin implementarea unui dicționar „visited”. Pentru a recupera soluția completă cum ajung la o stare finală, fiecare element din beam este un tuplu în starea hărții, lista cu secvența de stări de la start până la starea actuală și numărul de mutări (care e lungimea listei de secvențe cu stări dar am preferat să o rețin separat din rațiuni mai degrabă de lizibilitate).

Pentru fiecare candidat verific daca nu a mai fost vizitat si calculez scorul euristic alegând euristica mai mare oferindu-mi mi o stare mai promițătoare.

Euristici folosite

Ca si euristici plec de la 2 idei. O prima idee de euristica este ca in fiecare nod sa am distanta Manhattan pana la cea mai apropiata cutie + distanta de la cea mai apropiata cutie pana la ținta. O alta idee este am 2 tipuri de funcții euristice. Când am o cutie sa calculez doar distanta pana la target iar când nu am cutie sa calculez distanta pana la cea mai apropiata cutie. Fiind o matrice voi folosit in cadrul euristicilor doar distanta Manhattan. O alta idee ar fi ca pentru idee Manhattan sa prioritizeze cu o pondere cele 2 distante de la player la cutie si de la cutie la țintă.

De asemenea, m am mai gândit sa adaug si euristica misplace_box care numărără cate cutii nu sunt la locul lor. Este slaba dar am pus o pentru a observa grafic si din punct de vedere statistic ce rezultate am.

Tabel ce reprezintă numărul de pași un funcție de euristica si hartă

LRTA *

	Easy1	Easy2	Medium1	Medium2	Hard1	Hard2	Large1	Large2	Super_hard1
Manhattan	92	18	198	4681	2093	4681	2802	9300	568
Hungarian	340	18	206	5564	679	786	1225	99009	369
BFS_static	214	18	45	8053	308	417	921	75685	1668
BFS_heuristic	210	18	37	10059	308	529	887	27427	1636
Greedy	340	18	206	9037	679	1146	1225	93513	316
Advanced	210	18	45	5997	308	529	887	145831	2478
BFS_Hung	150	18	45	5997	64	1039	1209	143675	2478
Misplace_box	1048	578	18793	24097	10632	1420	109649	1000001	625765
Box_to_goal	1016	550	2359	3716	803	1783	16530	67152	1604

Beam Search

	Easy1	Easy2	Medium1	Medium2	Hard1	Hard2	Large1	Large2	Super_hard1
Manhattan	18	10	12	37	31	91	162	116	40
Hungarian	18	10	12	29	38	42	26	33	38
BFS_static	18	10	31	36	26	46	12	24	40
BFS_heuristic	18	10	12	24	31	36	26	46	40
Greedy	18	10	12	29	38	42	26	33	41
Advanced	18	10	12	24	31	36	26	46	40
BFS_Hung	18	10	12	24	31	36	26	46	40
Misplace_box	30	10	35	28	31	132	54	131	57
Box_to_goal	18	10	12	37	31	91	162	116	40

Observații:

- **Euristică “Misplace_box”** (numărul de cutii încă neplasate) e cea mai slabă: pe hărți mari (Large2, Super_hard1) produce sute de mii—milioane de pași, deci aproape inutilă.
- **Manhattan** este acceptabilă pe hărți mici dar crește foarte mult pe hărți mari.
- **Euristicile cu BFS** au îmbunătățiri semnificative pe testele mari.
- **Beam search** neutralizează diferențele între euristici având rezultate foarte apropiate

Descrieri ale euristicilor Folosite:

Manhattan – calculez distanța de la de la cutii la pozițiile lor țintă și o aleg pe cea mai apropiată pe cea mai bună și nu iau în considerare obstacolele.

Misplace_box- numără câte cutii nu și-au atins încă ținta

Box_to_goal_heuristic- estimează cât de aproape sunt toate cutiile de țintele lor, calculând pentru fiecare cutie distanța minimă până la o țintă și însumând aceste valori. Suportă atât distanța Manhattan, cât și Euclidiană. O sumă mai mică indică o stare mai apropiată de rezolvare.

Greedy - Construiește o potrivire cutie-țintă “pas cu pas”, alegând mereu cea mai apropiată cutie de o țintă nealocată. După ce am potrivit cutiile calculez distanța Manhattan a jucătorului până la cea mai apropiată cutie. Dacă numărul de cutii este mic poate fi folosită fiind un compromis între viteză și calitate.

Euristicile au de mai jos au fost preluate din anumite studii/lucrări cu scopul de a compara eficacitatea lor [3].

Hungarian- Folosește algoritmul Hungarian ca să găsească matching-ul global optim între cutii și ținte, pe baza distanțelor Manhattan, apoi adaugă la rezultat și pașii necesari pentru ca jucătorul să ajungă la prima cutie. Construiesc o matrice de costuri cu distanțele Manhattan între cutii și ținte după care le grupez astfel încât să minimizez suma totală a costurilor utilizând linear_sum_assignment.

BFS- Pre-procesezi o singură dată harta pentru fiecare țintă, cu un BFS care calculează distanțe “reale” (ignorând doar obstacolele statice) de la orice poziție din hartă la acea țintă. Apoi, la fiecare evaluare a stării, citești pur și simplu din aceste matrice. Pentru fiecare cutie parcurgi și alegi minimul dintre țintă și cutie după care însumez aceste distanțe și adaug distanța Manhattan până la cea mai apropiată cutie. BFS-ul a fost realizat cu sursa în fiecare țintă.

BFS_heuristic - Calculezi dinamic, la fiecare stare, distanțele reale (prin BFS) atât pentru mutarea cutiilor, cât și pentru deplasarea jucătorului, fără cache. BFS multi-țintă

pornind simultan din toate țintele după care BFS de la jucător(px,py) până la orice cutie după care returnez distanțele

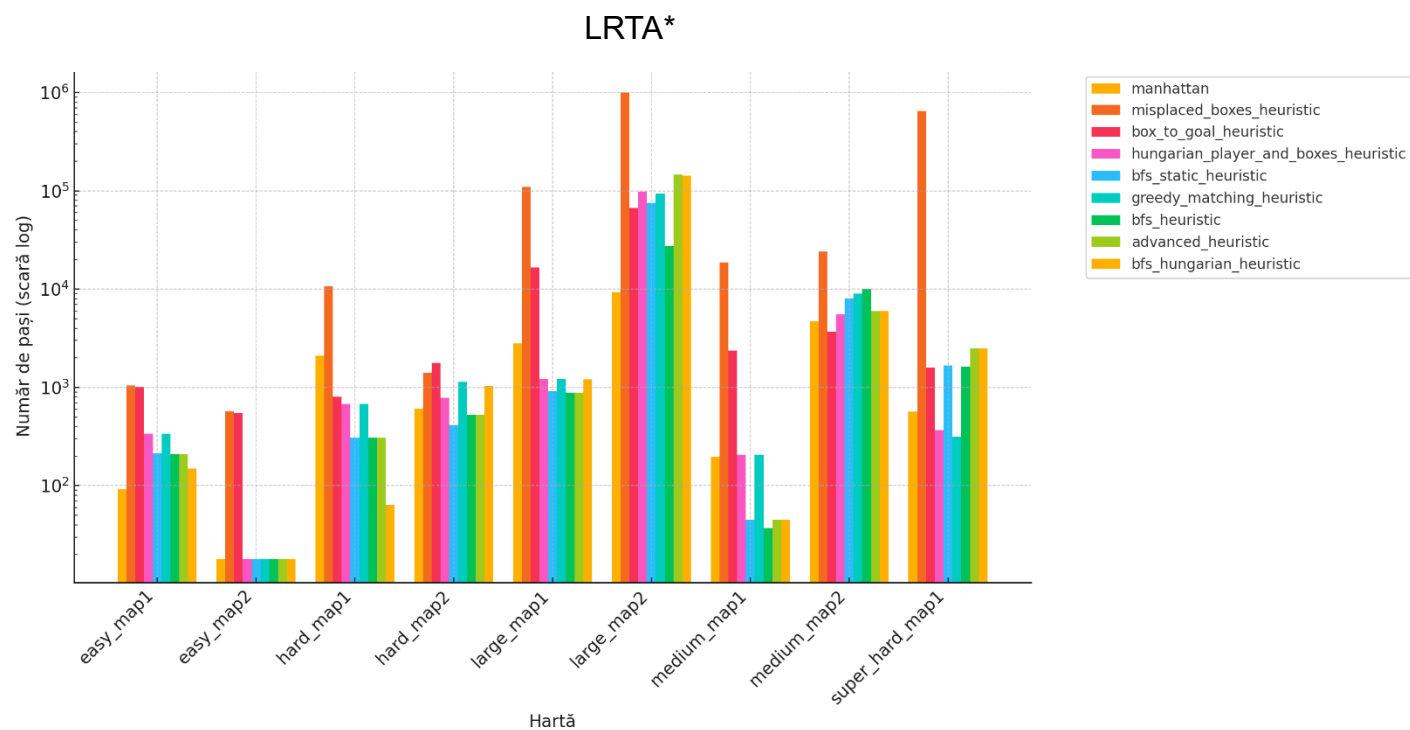
Advanced - Combină elemente din bfs_heuristic cu detectare de dead-lock și penalizări mari, plus o parte Manhattan pentru jucător. Pentru fiecare cutie care **nu e** deja pe țintă, verifici dacă e blocată în colț de două ziduri perpendiculare → dacă da, returnezi ∞ pentru fiecare cutie, citești din dist_map[box.x][box.y]. Dacă e inf, adaugi penalitate mare (1e6).

BFS_Hung - Este cea mai sofisticată: combină dead-lock, distanțe reale (= BFS), și matching optim pentru cutii, plus deplasare jucător. Cea mai **bine informată** euristică admisibilă, dar și cea mai **lentă** și consumatoare de memorie.

Comparație euristici

Am făcut o comparații grafice după diverse metrice rulând euristicele pe algoritmi LRTA* si Beam Search.

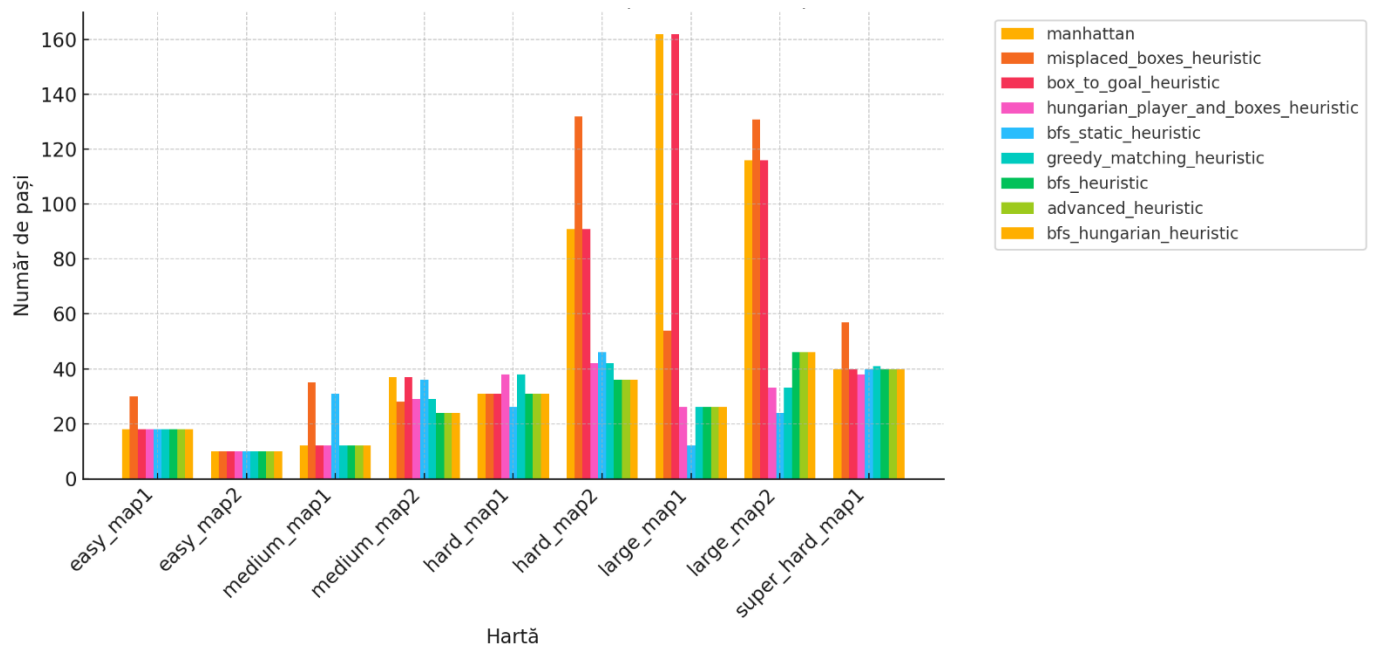
Comparație pe baza numărului de pași:



Observații:

- **Euristica misplaced_boxes** e departe de a fi eficienta.
- Pe large_map2 euristica misplaced_boxes ajunge la 10^6 pasi in timp ce cele cu bfs sunt la 10^4 – 10^5
- Exista o creștere exponențială pe hărți mari

BEAM SEARCH



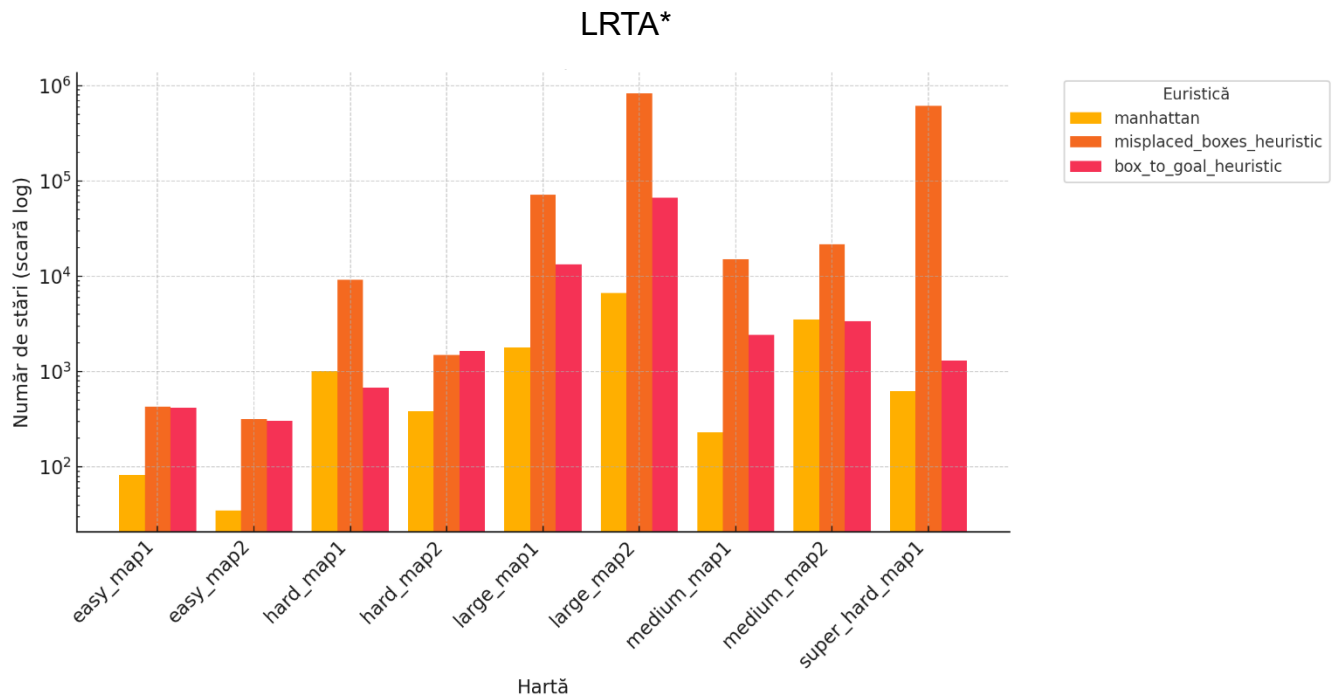
Observații:

- Cele mai grele hărți (large_map1, large_map2) trec de la zeci/ sute de mii de pași (LRTA*) la **sub 160 pași** maxim.
- Se păstrează cam aceeași ordine a euristiciilor din punct de vedere al eficienței
- Se observă o performanță uniformă pe testele easy și medium fiind între 10 și 35 de pași iar pe mapa super_hard_map1 variația între euristicii se comprimă având aproximativ același număr de pași.

Concluzii:

În esență, LRTA* evidențiază diferențe uriașe între euristicii (în special pe hărți mari), în timp ce beam search menține aceleași ierarhii, dar cu ordinea de mărime redusă de câteva mii de ori a numărului de pași și cu o variabilitate mult mai mică.

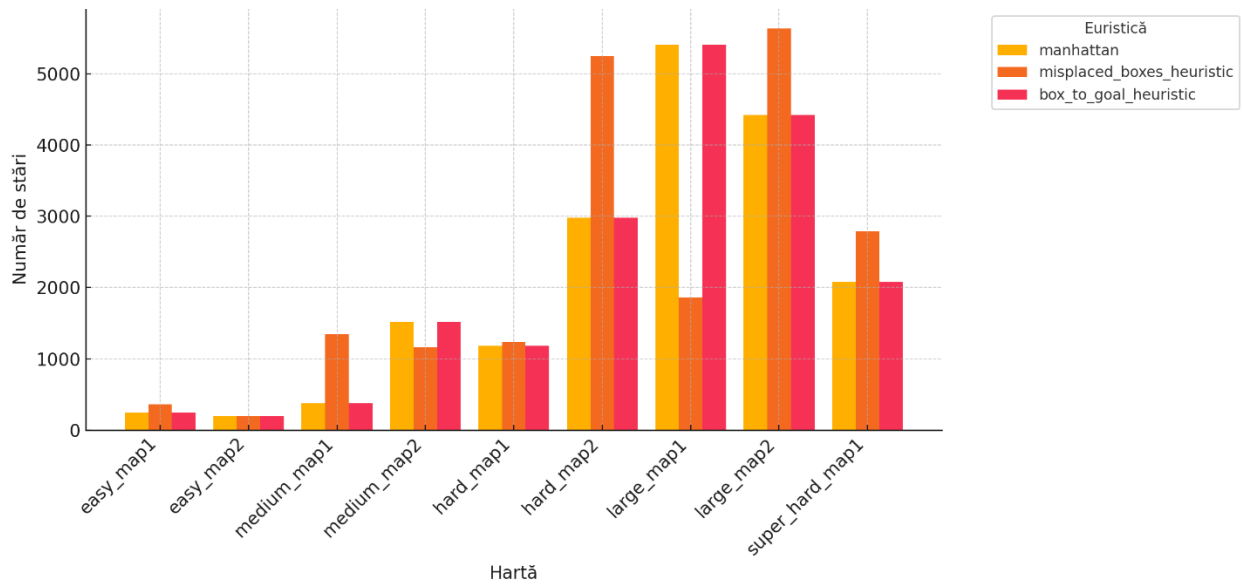
Comparație grafică în funcție de numărul de stări expandate



Observații:

- Pe hărți mari **misplaced_boxes** explodează la zeci de mii (către 10^5) de stări expandate (medium_map1, medium_map2), apoi sute de mii–un milion pe large_map2 și super_hard_map1.
- Pe hărțile de dificultate medie (medium_map1, medium_map2), Manhattan e cu un ordin de mărime mai bună decât misplaced și puțin sub box_to_goal, dar toate rămân sub $\sim 10^4$ pași.

BEAM SEARCH



Observații:

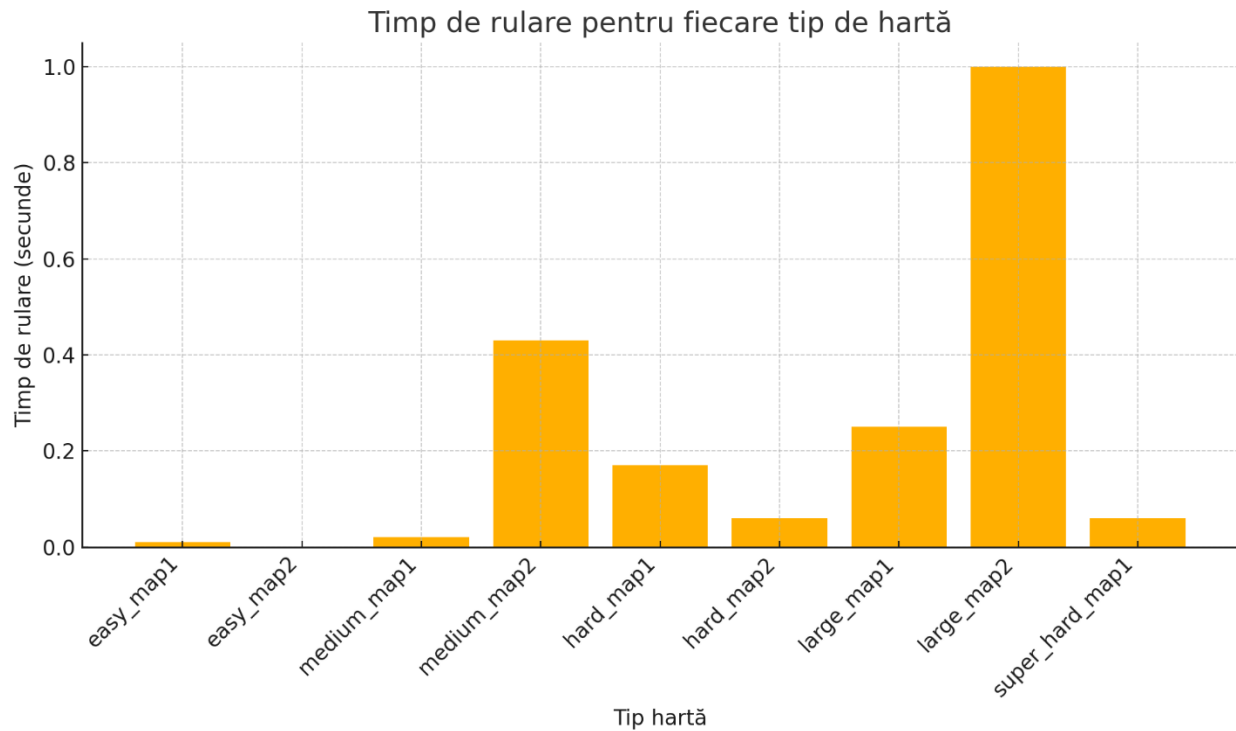
- Numărul de stări expandate se comprimă drastic pentru toate euristicile
- **Manhattan** și **box_to_goal** sunt aproape egale pe majoritatea hărților, cu o ușoară ușurință în favoarea lui Manhattan.
- **Misplaced_boxes** este cea mai slabă dar nu trece de 6000 de stări

Concluzii:

- Ordinea euristicilor rămâne aceeași
- **Beam Search** reduce absolut numărul de stări cu **ordinea de mărime a zecilor-sutelor de ori**, față de LRTA*, și atenuează diferențele relative.
- **Importanța euristicii** persistă: chiar și în beam, o euristică mai slabă costă câteva mii de stări în plus pe hărți mari.

Comparație grafica in funcție de timpul de rulare

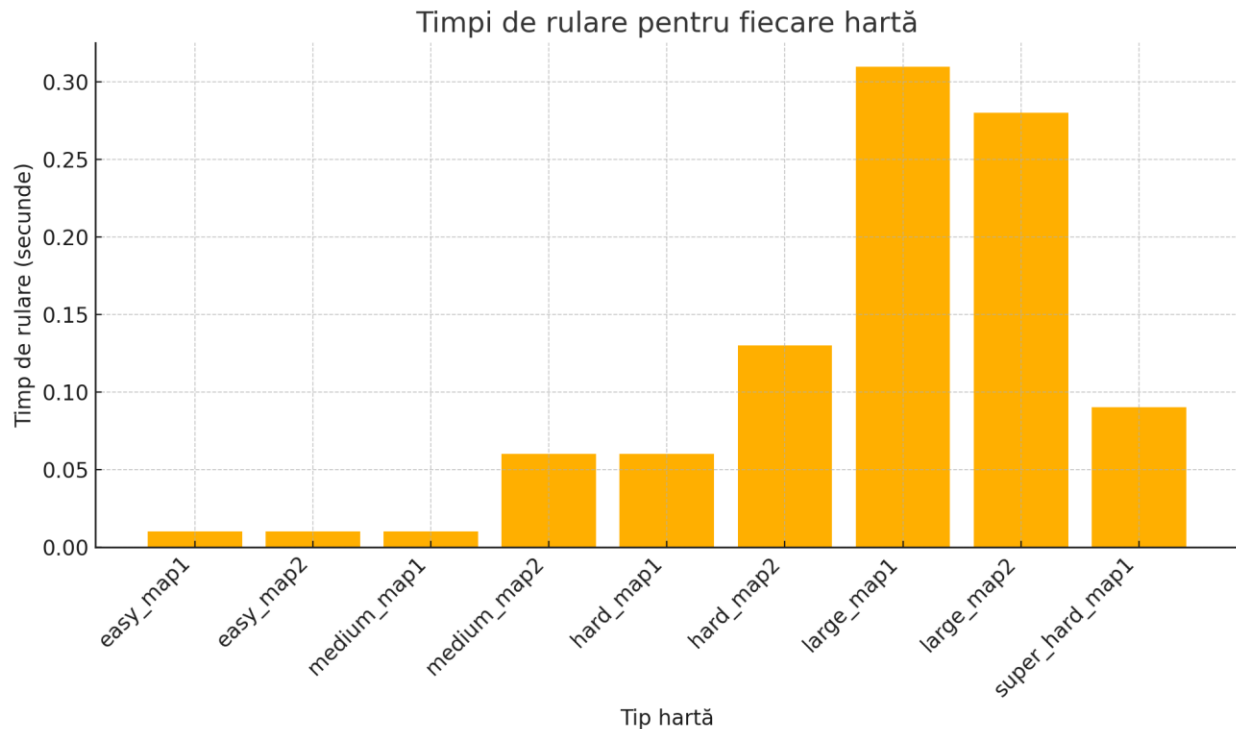
LRTA*



Observații:

- Se observa pe hărțile medii saltul de la medium map1 la 2 întrucât spațiul de căutare este mai mare si este saltul de la 0.02 s la 0.43 s.
- De asemenea pe large map2 observam aceeași diferența mare între ea si large map1.

BEAM SEARCH



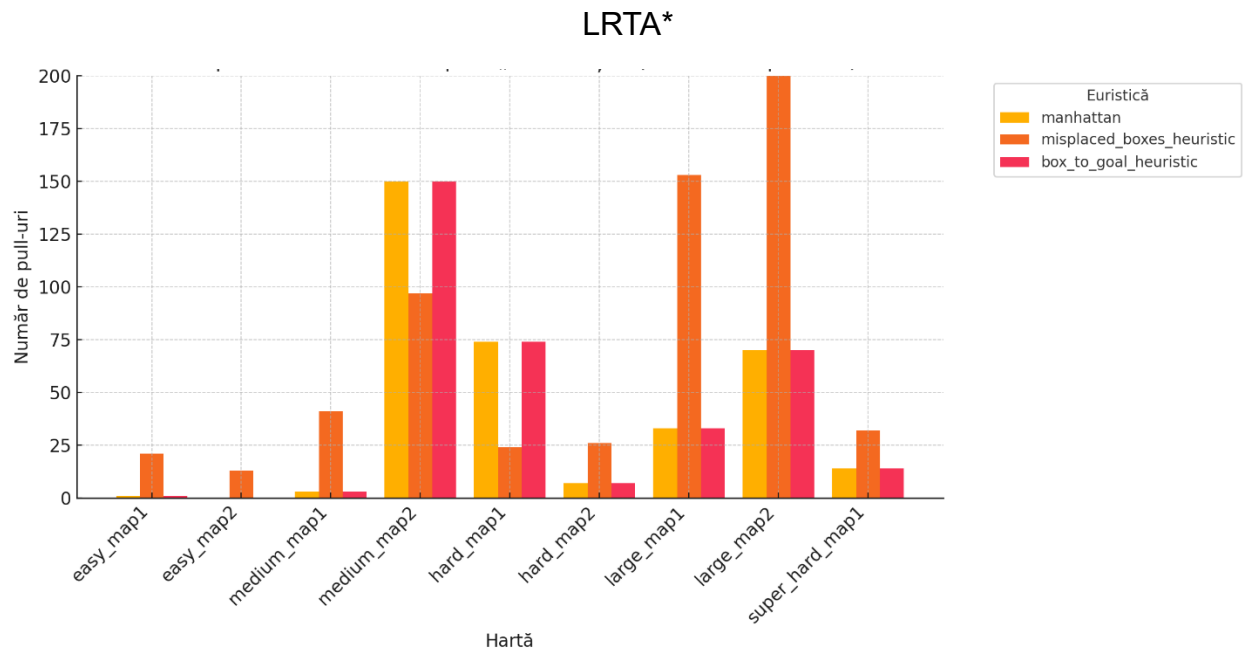
Observații:

- Avem timp foarte mici de pana in 0.05 s pe testele ușoare si medii.
- Pe super_hard_map1 este de 3 ori mai rapid fata de LRTA*

Concluzii:

- Pe majoritatea hărților medii și mari, beam reduce timpul de până la 3–5× față de LRTA*
- LRTA* are variații mari de timp
- Pentru hard_map2, large_map1 și super_hard_map1, beam_search plătește uneori costul de taiere depășind ușor LRTA*

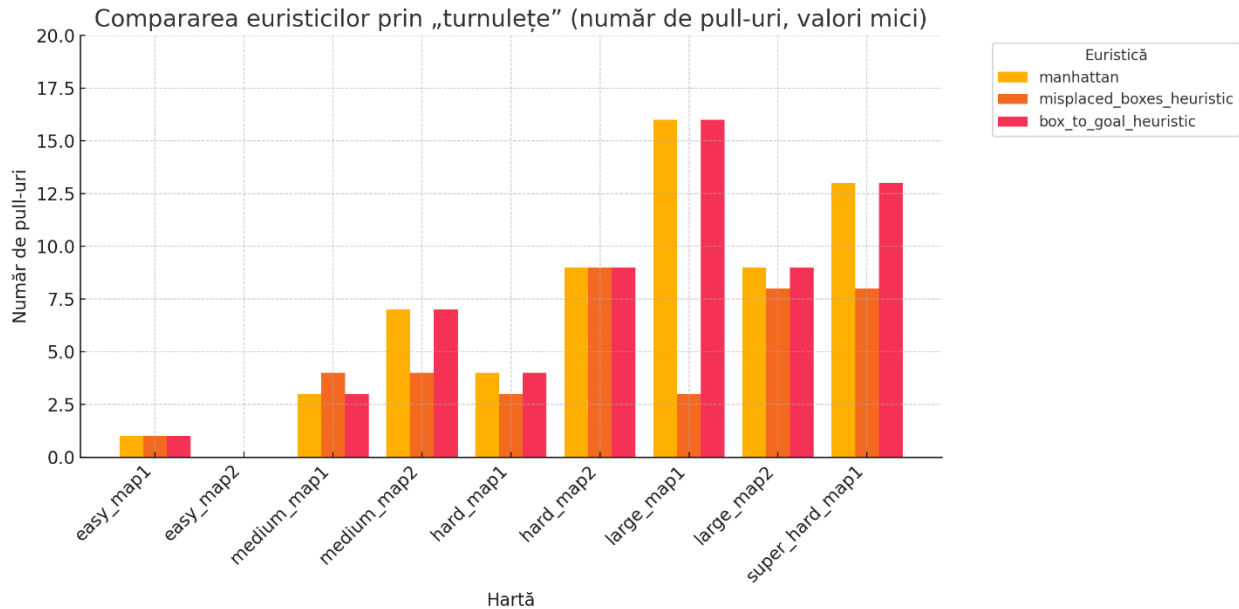
Comparație grafica in funcție de numărul de operații de tip pull



Observații:

- **Misplaced_boxes** necesită constant cu zeci sau chiar sute de pull-uri în plus față de celelalte două euristici
- **Manhattan și box_to_goal** au profile aproape identice de pull-uri

BEAM SEARCH



Observații:

- Pull-urile se uniformizează drastic: chiar și **misplaced_boxes** cade la un număr comparabil (3–9), față de 14–200 în LRTA*.
- Valorile maxime de pull-uri se găsesc la folosirea **Manhattan** și **box_to_goal** pe hărți mari, dar la valori sub 20.

Concluzii:

- **Beam Search reduce masiv numărul de pull-uri** pentru toate euristiciile, îndeosebi pentru **misplaced_boxes**, care în LRTA* era de câteva zeci–sute, iar în beam stă sub 10 pe majoritatea hărților.
- **Manhattan** și **box_to_goal** au performanțe foarte apropiate în ambele algoritme, cu un număr minim de pull-uri aproape constant.
- **Importanța euristicii slabe** (**misplaced_boxes**) persistă în LRTA*, dar beam search atenuează aproape complet acest dezavantaj, comprimând toți parametrii la un nivel redus și uniform de pull-uri.

Referințe:

- [1] - https://turing.cs.pub.ro/blia_2003/Real-time_search_1.htm
- [2] - <https://www.geeksforgeeks.org/introduction-to-beam-search-algorithm/>
- <https://www.askpython.com/python/examples/beam-search-algorithm>
- [3] - <https://www.sciencedirect.com/science/article/pii/S0004370215000867#:~:text=Heuristics%20for%20Sokoban,instance%20with%20only%20one%20stone>
- <https://timallanwheeler.com/blog/2022/01/19/basic-search-algorithms-on-sokoban/>
- <https://timallanwheeler.com/blog/2022/01/23/sokoban-reach-and-code-performance/>
- <https://stackoverflow.com/questions/4237462/sokoban-solver-tips>